Updates to readme
Iovino, David authored just now

Name	Last commit	Last update
collections	cleanup 08-07-24	13 hours ago
modules	cleanup 08-07-24	13 hours ago
templates	update 2024-07-16	3 weeks ago
.gitignore	cleanup 08-07-24	13 hours ago
README.rst	Updates to readme	just now
cptools.py	cleanup 08-07-24	13 hours ago
requirements.txt	cleanup 08-07-24	13 hours ago

README.rst

CPTOOLS

This code has been made available for the early development of **content primitives** for use with **Intelligent Content Briefing Framework (ICBF)**. Both content primitives and ICBF were developed as part of the **Content Modernization Workstream**. The project goal was to drive efficiency gains in the creation of long form content by incorporating process improvements, automation and GenAI.

Getting started

pip install -r requirements.txt

Operations

Create a collection (available)

The following command creates an environment for organizing source content for a content primitive.

```
cptools create <collection name>
```

Build a content primitive (available)

The following command will create the content primitive from a collection.

```
cptools build <collection name>
```

Analyze a source document (future)

When updating an existing document, comments can be used to identify the changes needed. The following command will evaluate those comments and pre-populate the collection.

```
cptools analyze <collection name>
```

Check the consistency of a collection (future)

The following command will evaluate all components within a collection and flag any inconsistencies.

```
cptools lint <collection name>
```

The following files and directories are created in response to `cptools create <collection name>`.

- **.cpdata**: Contains a small sqlite database that is used during the build process.
- **code**: Place text-based code snippets in this directory. Use the following naming convention `code_001.txt`.
- **content_primitive**: This directory is populated in response to `cptools build <collection name>` and represents a complete ICBF-ready content primitive.
- **descriptions**: Place image and table descriptions in this directory. Use the following naming convention `description_image_001.txt` or `description_table_001.txt`.
- **images**: Place images in this directory. Use the following naming convention `image_001.png` or `image_001.jpg`.
- **journals**: Create your journals in this directory. Use the following naming convention `journal_001.txt`.
- **misc**: Place any supplemental files that you want to keep with the collection here. This directory is ignored during the build process.
- **source**: For updates to existing technical papers, place the source .docx file here.
- **tables**: Place tables in this directory. Use the following naming convention `table_001.html`.
- **text**: Not currently used.
- **manifest.txt**: This file is used to capture chapter-level actions that will drive ICBF.
- **metadata.yml**: This file is used to capture metadata for this collection.
- **references.yml**: This file is used to capture Infohub references that ICBF can reference.

Journal files

Journal files are the heart of content primitives. The examples below will illustrate how to create them.

Example 1: The start of a new chapter

The two hash symbols before **Introduction** indicate this section will be **Heading 2**. ICBF will automatically add the chapter title (defined in the manifest) as **Heading 1**. Define the contents of this section in outline form and let ICBF do the creative writing. Use an outline format that works best for you.:

```
## Introduction
- Benefits
  + Increased workload mobility.
  + Cross-site load balancing.
  + Fast recovery at a remote site within metro distance.
  + Disaster avoidance for planned downtime.
```

Example 2: Adding images, tables and code snippets

Use the following syntax to add images, tables and code snippets. For images and tables the build process will automatically add the appropiate label from the descriptions directory. Labels are not included for code snippets.:

```
<<figure: image-022.png>>
<<table: table-002.html>>
<<code: code-001.txt>>
```

Example 3: Replace or remove images and tables existing chapter

Use the following syntax to replace or remove images and tables.:

```
- Replace figure 9 with <<figure: image-023.png>>
- Replace table 8 with: <<table: table-003.html>>
- Remove figure 49
```

Example 4: Add a new section to an existing chapter

Use the following syntax to add a new section an existing chapter.:

```
## Red Hat documentation
The following Red Hat documentation provides information related to Red Hat Enterprise Linux and Red Hat High-Availa
- Red Hat Enterprise Linux 8 Documentation: https://access.redhat.com/documentation/en-us/red_hat_enterprise_linux/8
- Configuring and managing high availability clusters: https://access.redhat.com/documentation/en-us/red_hat_enterp
```

The manifest file defines the actions (also call pre-stages prompts) for content primitives. The example below illustrates the syntax.:

```
# Manifest file represents a list of actions for ICBF. Each action acts as pre-staged prompts within ICBF
#
# Each line in this file represents an action and will be paired with a journal file"
# Each line needs to use the following format:
# 'chapter title': 'Action to be taken' :: 'journal filename'
# The journal filename is create/updated automatically
# The only time you should add something after the '::' is if this action is atomic and not journal is needed
#
Overall: Remove any reference to PowerStore X model :: None :: None
Overall: Remove/replace figures:: journal-020.txt :: None
Overall: Remove/replace tables:: journal-022.txt :: None
Executive summary | Overview: Update existing text to reflect new and extended PowerStoreOS 4.0 Metro Volume featur
Introduction: Update existing text to reflect new and extended PowerStoreOS 4.0 Metro Volume features:: journal-001
Metro Volume: Update existing text to reflect new and extended PowerStoreOS 4.0 Metro Volume features:: journal-001
Metro Volume support for Microsoft: Add new chapter:: journal-010.txt :: None
Metro Volume support for Linux: Add new chapter:: journal-012.txt :: None
References: Add new section:: journal-198.txt :: None
Appendix A: Add new chapter:: journal-199.txt :: None
```

Metadata file

The metadata file defines the metadata for content primitives. The example below illustrates the syntax.:

```
---
project:
        name: "Metro Volume whitepaper update"
        description: "Updating the existing Metro Volume whitepaper with new features"
        primary_effort: "development"
output:
        category: "technical_paper"
        subcategory: "whitepaper"
        tone: "formal"
        type: "update"
        update_ref_id: ""
        name: "Dell PowerStore: Metro Volume"
        description: "This white paper provides in-depth coverage of Dell PowerStore Metro Volume and its features"
...
```

Reference file

The metadata file defines references to be used for additional context. The example below illustrates the syntax.:

```
    ---
    ref-001:
    status: public
    location: infohub
    details:
            type: url
            path: "https://infohub.delltechnologies.com/en-us/t/deployment-best-practices-for-oracle-databases-with
...
```